UNIVERSITÀ DI BOLOGNA



School of Engineering

Master Degree in Automation Engineering

Optimal Control and Reinforcement Learning

Design and Implementation of an Optimal Control Framework for 2-DOF Flexible Robot Arm using LQR and MPC Trajectory Tracking

Professor: **Giuseppe Notarstefano**

Students:
Hanif Hidayat
Muhanad Albahloul Alajeeli Souysi

Academic year 2025/2026

Abstract

This project explores the design and implementation of an optimal control framework for a two-degree-of-freedom (2-DOF) flexible robot arm characterized by nonlinear dynamics. The study begins with the derivation of the system's continuous-time equations of motion, which are subsequently discretized using a fourth-order Runge-Kutta (RK4) scheme to ensure numerical stability and accuracy. Two primary objectives are addressed: optimal trajectory generation and trajectory tracking. Trajectory generation is performed using a Newton-based optimization method. The performance of discontinuous "step" references against "smooth" cubic polynomial references are compared, demonstrating that smoothness significantly reduces control effort and tracking error. For the tracking phase, two distinct strategies are compared: a time-varying Linear Quadratic Regulator (LQR) and a Model Predictive Control (MPC) scheme. While LQR provides robust feedback against initial perturbations, MPC demonstrates superior performance by explicitly handling input saturation and state constraints. The results are validated through numerical simulations and 2D animations, confirming the physical feasibility and efficiency of the proposed control architectures.

Contents

Introduction **6**

1 Task 0: Problem Setup and Dynamics Discretization **7**

2 Task 1: Trajectory Generation with Step Reference **10**
2.1 Method 10
2.2 Results 13

3 Task 2: Trajectory Generation using Smooth Reference **17**
3.1 Method 17
3.2 Results 17

4 Task 3: Trajectory Tracking via LQR **20**
4.1 Trajectory Tracking Formulation 20
4.1.1 System Linearization 20
4.1.2 Riccati Recursion 21
4.1.3 Feedback Control Law 21
4.1.4 Backward Recursion Algorithm 21
4.1.5 Forward Simulation 22
4.2 LQR Tracking Results 22
4.3 Tracking Error Analysis 22

5 Task 4: Trajectory Tracking via MPC **24**
5.1 Model Predictive Control Framework 24
5.1.1 Optimization Problem Formulation 24
5.1.2 Implementation Details 25
5.1.3 Constraints 25
5.1.4 Optimization Output 25
5.2 MPC Tracking Results: Unconstrained 26
5.3 MPC Tracking Results: Constrained 27
5.4 MPC Tracking Error 27

6 Task 5: Animation and Visualization **29**
6.1 Animation Framework 29
6.2 Observations 29

Conclusions **31**

Introduction

The control of flexible robotic structures represents a significant challenge in modern automation. This project focuses on a 2-DOF flexible robot arm, where the goal is to transition the system between different angular configurations while minimizing a cost function of state error and control effort.

The complexity of the system arises from its nonlinear Lagrangian dynamics, which include position-dependent inertia matrices, Coriolis forces, and gravitational terms, further complicated by internal stiffness and viscous friction. To manage this complexity, the project adopts a modular approach: first establishing a reliable discrete-time model, then solving the "open-loop" trajectory optimization problem, and finally designing "closed-loop" controllers to handle disturbances and constraints.

By utilizing modern computational tools such as `CasADi` for algorithmic differentiation and `CVXPY` for convex optimization, this work demonstrates the transition from classical optimal control theory to practical, high-performance tracking algorithms like LTV-LQR and MPC.

Chapter 1

Task 0: Problem Setup and Dynamics Discretization

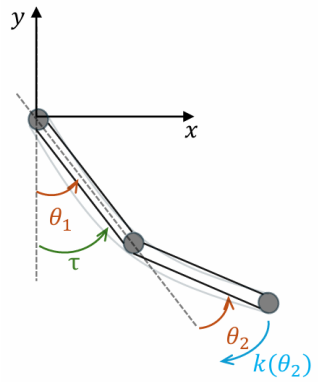


Figure 1.1: Flexible robot arm model

This project aims to design an optimal trajectory of a flexible robot arm with two degrees of freedom, as shown in Figure 1.1. The dynamics (in a continuous time setup) are expressed in the following equation:

$$M(\theta_1, \theta_2) \begin{bmatrix} \ddot{\theta_1} \\ \ddot{\theta_2} \end{bmatrix} + C(\theta_1, \theta_2, \dot{\theta_1}, \dot{\theta_2}) \begin{bmatrix} \dot{\theta_1} \\ \dot{\theta_2} \end{bmatrix} + F \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} + G(\theta_1, \theta_2) = \begin{bmatrix} \tau \\ 0 \end{bmatrix}$$

Where:

$$M = \begin{bmatrix} I_1 + I_2 + l_{c1}^2 m_1 + m_2(l_1^2 + 2l_1 l_{c2}\cos(\theta_2) + l_{c2}^2) & I_2 + l_{c2} m_2 (l_1 \cos(\theta_2) + l_{c2}) \\ I_2 + l_{c2} m_2 (l_1 \cos(\theta_2) + l_{c2}) & I_2 + l_{c2}^2 m_2 \end{bmatrix}$$

$$C = \begin{bmatrix} -l_1 l_{c2} m_2 \dot{\theta}_2 \sin(\theta_2) & -l_1 l_{c2} m_2 (\dot{\theta}_1 + \dot{\theta}_2) \sin(\theta_2) \\ l_1 l_{c2} m_2 \dot{\theta}_1 \sin(\theta_2) & 0 \end{bmatrix}$$

$$G = \begin{bmatrix} gl_{c1}m_1 \sin(\theta_1) + gm_2(l_1 \sin(\theta_1) + l_{c2} \sin(\theta_1 + \theta_2)) \\ gm_2 l_{c2} \sin(\theta_1 + \theta_2) + 3k \sin(\theta_2) \cos^2(\theta_2) \end{bmatrix}$$

$$F = \begin{bmatrix} f_1 & 0 \\ 0 & f_2 \end{bmatrix}$$

The parameter setup of the dynamics is shown in Table 1.1. The dynamics model can be simplified into the following problem statement

$$\ddot{q} = M(q)(u - C(q, \dot{q})\ \dot{q} - F\dot{q} - G(q))$$

with $q = [\theta_1\ \theta_2]^\top$ and $u = [\tau\ 0]^\top$. In the file `dynamics.py`, a Python library `CasADi` is utilized to express the symbols.

m_1	2
m_2	2
l_1	1.5
l_2	1.5
l_{c1}	0.75
l_{c2}	0.75
I_1	1.5
I_2	1.5
g	9.81
f_1	1.0
f_2	1.0
k	0.6

Table 1.1: Parameter setup

After defining the dynamics, the discretization is done by implementing the Runge-Kutta method of degree 4 (RK4). First, the problem can be redefined as $\dot{x} = f(x_t, u_t)$ with $x_t = [\theta_1\ \theta_2\ \dot{\theta_1}\ \dot{\theta_2}]$.

The algorithm of RK4 will approximate the dynamics with the following rules:

$$\begin{cases} k_1 = f(x_t, u_t) \\ k_2 = f(x_t + \frac{dt}{2} k_1, u_t) \\ k_3 = f(x_t + \frac{dt}{2} k_2, u_t) \\ k_4 = f(x_t + dt\ k_3, u_t) \\ x_{t+1} = x_t + \frac{dt}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{cases}$$

The results show that the RK4 discretization can approximate the non-linear dynamics due to the viscous friction matrix F, better than Euler's. However, the RK4 discretization cannot compute the Jacobian $\partial f(x_t, u_t)/\partial x$ and $\partial f(x)/\partial u$ directly. To save the computation, the Jacobian function provided by the `CasADi` library is used.

Chapter 2

Task 1: Trajectory Generation with Step Reference

2.1 Method

Before generating the trajectory, a numerical method to calculate the system's equilibria is needed. In this project, Newton-Raphson root finding from the `scipy` library is utilized. From the dynamics model, the equilibrium requires the state $\dot{\theta_1}, \dot{\theta_2}, \ddot{\theta_1}, \ddot{\theta_2} = 0$. Therefore, the dynamics model can be expressed as the following equation. Revisiting the model equation described in Chapter 1, the system in equilibrium can be expressed as follows.

$$G(\theta_1, \theta_2) = \begin{bmatrix} \tau \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} gl_{c1}m_1 \sin(\theta_1) + gm_2(l_1 \sin(\theta_1) + l_{c2} \sin(\theta_1 + \theta_2)) \\ gm_2 l_{c2} \sin(\theta_1 + \theta_2) + 3k \sin(\theta_2) \cos^2(\theta_2) \end{bmatrix} = \begin{bmatrix} \tau \\ 0 \end{bmatrix}$$

The problem of root finding is that to find the angle of θ_2 and the required input τ given the desired angle of θ_1. The equation above can be redefined to describe the root-finding problem better as the following.

$$\begin{bmatrix} G_1(\theta_1, \theta_2) - \tau \\ G_2(\theta_1, \theta_2) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$H(z) = 0$$

where $z = [\theta_2 \ \tau]^\top$.

The update rule to find the root of the equilibrium is then formulated as follows.

$$z_{k+1} = z_k - J(z_k)^{-1} \ H(z_k)$$

where $J(z_k)$ is the Jacobian of the residual matrix $H(z_k)$, which can be formulated as $J(z) = \frac{\partial H}{\partial z} = \begin{bmatrix} \frac{\partial H_1}{\partial \theta_2} & \frac{\partial H_1}{\partial \tau} \\ \frac{\partial H_2}{\partial \theta_2} & \frac{\partial H_2}{\partial \tau} \end{bmatrix}$.

In the code, the problem is solved using the `Numpy` library's method `linalg.solve()`. The problem is redefined as $J(z_k)\ \Delta z = -H(z_k)$. Hence, the update rule is now as follows.

$$z_{k+1} = z_k + \Delta z$$

The algorithm is repeated until $||H(z_k)||_2 < \ 1 \times 10^{-9}$.

As Task 1 required, a step reference path is generated by creating a new function in `reference_trajectories.py`. First, the initial and final configurations are identified by the following conditions.

$$\theta_{1,ref}(t) = \begin{cases} 0^\circ & \text{for } 0 \leq t < T/2 \\ 30^\circ & \text{for } T/2 \leq t < T \end{cases}$$

By using the Newton-Raphson root finding from the function `equilibria.py`, it is known that the second joint configuration θ_2 and the required input $u(t) = \tau$ are as follows.

$$\theta_{2,ref}(t) = \begin{cases} 0^\circ & \text{for } 0 \leq t < T/2 \\ -30^\circ & \text{for } T/2 \leq t < T \end{cases}$$

$$u_{ref}(t) = \begin{cases} 0 & \text{for } 0 \leq t < T/2 \\ 40 & \text{for } T/2 \leq t < T \end{cases}$$

After the two set points are decided, the trajectory generation can be formalized by minimizing the quadratic cost function as

$$\begin{aligned} J = \frac{1}{2} \sum_{t=0}^{T-1} [(x_t - x_{ref,\ t})^\top Q (x_t - x_{ref,\ t}) + \\ (u_t - u_{ref,\ t})^\top R (u_t - u_{ref,\ t})] + \\ \frac{1}{2} (x_t - x_{ref,\ T})^\top Q_f (x_t - x_{ref,\ T}) \end{aligned}$$

subject to $x_{t+1} = f(x_t, u_t)$. The weighting matrices $Q, R,$ and Q_f are described below:

- $Q = 0.1 \times \text{diag}[100, 0, 0.1, 100]$ that puts more priority on tracking θ_1 and $\dot{\theta}_2$.
- $R = 0.0001$ that penalizes the input error.

- $Q_f = Q$, which is the terminal error weight.

The Newton algorithm is implemented to minimize the cost function above. The function can be simplified by the following equation.

$$J^{(k)} = \sum_{t=0}^{T-1} \underbrace{l(x_t^{(k)}, u_t^{(k)})}_{\text{stage cost}} + \underbrace{l_f(x_T^{(k)})}_{\text{terminal cost}}$$

The performance is then evaluated by calculating the gradient of the terminal cost at $t = T$ expressed in the following equation.

$$\lambda_T^{(k)} = \nabla_{x_T} l_f(x_T^{(k)}) = Q_f(x_T^{(k)} - x_{ref,T})$$

The integration then goes backward for $t = T-1,\ T-2, \ldots,\ 0$ formulated as follows.

$$\lambda_t^{(k)} = A_t^T \lambda_{t+1}^{(k)} + \nabla_{x_t} l(x_t^{(k)}, u_t^{(k)})$$

The matrix $A_t = \frac{\partial f}{\partial x}\Big|_{x_t^{(k)}, u_t^{(k)}}$ is the state Jacobian that linearizes the system dynamics.

Now, the control input u is determined to see the effect on the total cost using the gradient of the Hamiltonian in the following equation.

$$\nabla_{u_t} J^{(k)} = B_t^T \lambda_{t+1}^{(k)} + \nabla_{u_t} l(x_t^{(k)}, u_t^{(k)})$$

The matrix $B_t = \frac{\partial f}{\partial u}\Big|_{x_t^{(k)}, u_t^{(k)}}$ is the input Jacobian, representing actuator effectiveness. The term $B_t^T \lambda_{t+1}^{(k)}$ captures the future consequences of the current action.

Recollecting the parameters obtained from the previous steps, an LTV-LQR approach is utilized to determine the descent direction by applying

$$[K_t, \sigma_t] = \text{LTV-LQR}(A_t, B_t, Q, R, q_t, r_t, Q_f, q_f)$$

where:

- q_t, r_t: First-order derivatives (gradients) of the cost.
- Q, R: Second-order derivatives (Hessian) of the cost.
- with the output:
  - K_t: Feedback gain matrix (corrects state deviations).
  - σ_t: Feedforward term (the descent direction/shift in control).

Since the system is non-linear, a full descent step might overshoot. A line search algorithm is used to find the appropriate scaling factor $\alpha^{(k)} \in (0, 1]$ by verifying the Armijo Condition:

$$J(u^{(k)} + \alpha^{(k)}\Delta u^{(k)}) \leq J(u^{(k)}) + c\alpha^{(k)}\nabla J^{(k)T}\Delta u^{(k)}$$

which checks the sufficient descent given the step size. The parameters in the step size selection are described below.

- $c = 0.5$
- $\beta = 0.7$ for updating the step size fulfilling the Armijo Condition
- $\alpha^{(0)} = 1$
- Maximum Armijo iterations: 20

The control law is updated the system to generate the trajectory for iteration $k + 1$.

$$u_t^{(k+1)} = u_t^{(k)} + K_t(x_t^{(k+1)} - x_t^{(k)}) + \alpha^{(k)}\sigma_t$$

The term $K_t(x_t^{(k+1)} - x_t^{(k)})$ allows the controller to adjust reactively if the new state trajectory deviates from the previous one, providing stability. u_t will be used to update the next state by the following equation.

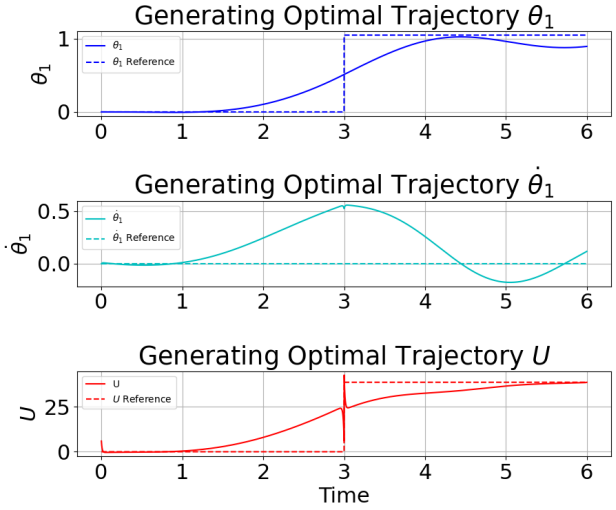
$$x_{t+1}^{(k+1)} = f(x_t^{(k+1)}, u_t^{(k+1)})$$

The algorithm terminates when the change in control input is negligible:

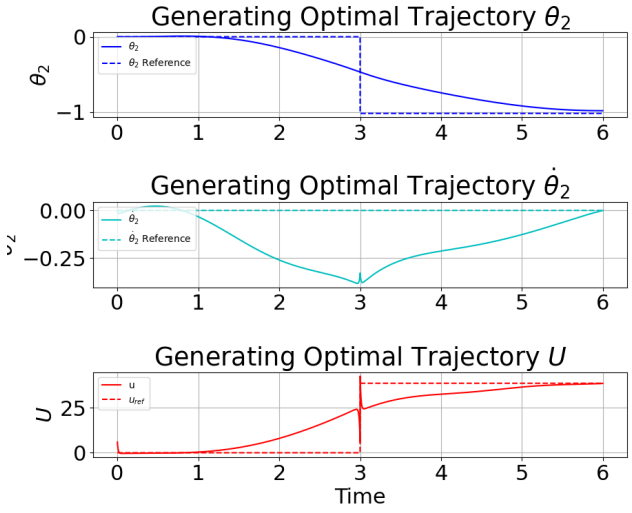
$$\sum_{t=0}^{T-1} \|\Delta u_t^{(k)}\|_2^2 < 10^{-6}$$

2.2 Results

The algorithm successfully finds the transition between the initial and terminal states, and the inputs generated by the step reference, as Figure 2.1. The cost function illustrated in Figure 2.2 implies a significant descent occurs up to $k = 2$, and the descent direction shows the later iterations have smaller gradient norms. From the figure, it is concluded that the generated trajectory leaves a noticeable error that prevents the state θ_1 from converging $\theta_{1,ref}$ within $T = 6$ seconds. The step size adjustments during the descent are shown in Figure 2.3.

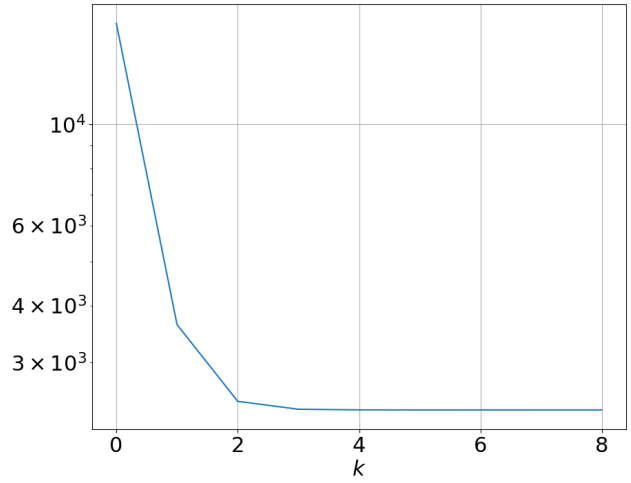


(a) q_1 trajectory plot

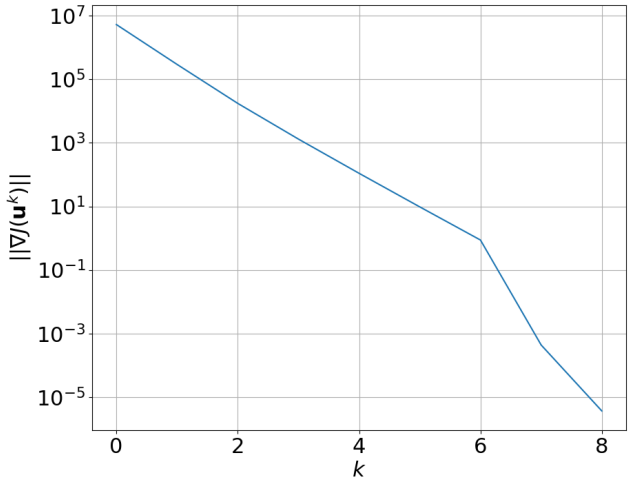


(b) q_2 trajectory plot

Figure 2.1: Trajectory generation with step reference

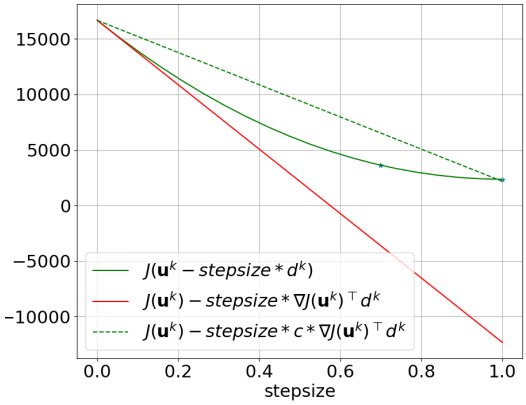


(a) Cost function

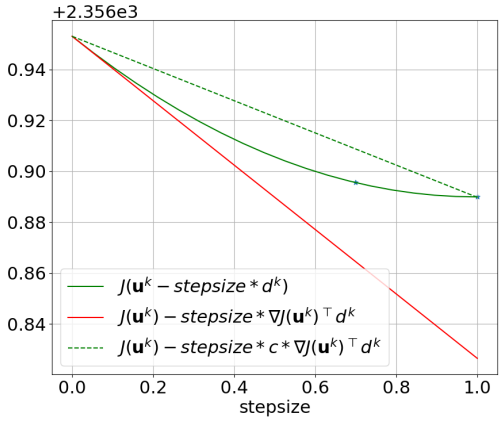


(b) Descent direction

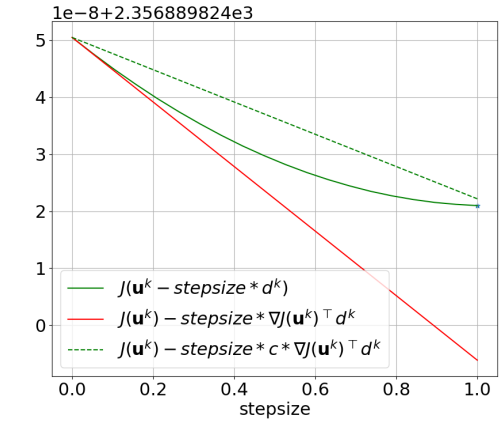
Figure 2.2: Cost function over k iterations with step reference



(a) Step size at $k = 0$



(b) Step size at $k = 4$



(c) Step size at $k = 9$

Figure 2.3: Step size selection using Armijo rule

Chapter 3

Task 2: Trajectory Generation using Smooth Reference

3.1 Method

The task requires generating a smooth reference for the robot arm to follow. A set of θ_1 intermediate points is introduced ($\theta_1 = \{0^\circ\ , 60^\circ, 30^\circ, 0^\circ\}$) for the robot to follow within $T = 12$ seconds. Then, these points are interpolated using a cubic polynomial equation $s(x) = 3x^2 - 2x^3$. The reference $\mathbf{x}_{ref}$ and $\mathbf{u}_{ref}$ are formulated as:

$$\begin{cases} \mathbf{x}_{ref}(x) = \mathbf{x}_{start} + s(x) \cdot (\mathbf{x}_{end} - \mathbf{x}_{start}) \\ \mathbf{u}_{ref}(x) = \mathbf{u}_{start} + s(x) \cdot (\mathbf{u}_{end} - \mathbf{u}_{start}) \end{cases}$$

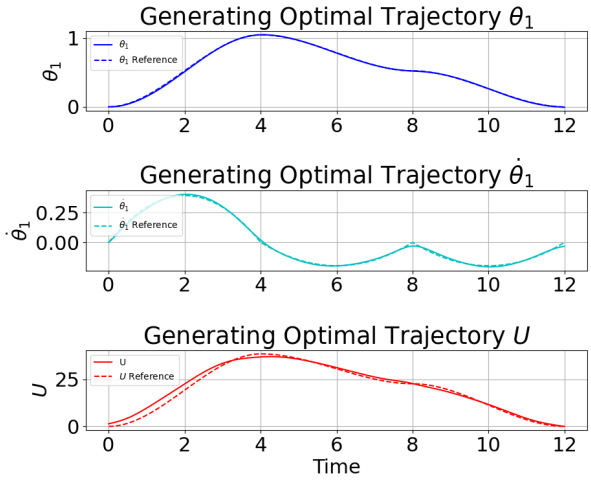
The second joint angle θ_2 and the control input u are obtained from the equilibrium computation by the function `equilibria.py`.
The next steps in this task are similar to Task 1, which are:

1. Forward cost evaluation
2. Backward costate integration
3. LTV-LQR descent
4. Armijo step size selection
5. Stop the algorithm when $\|\Delta u_t^{(k)}\|_2^2 < 10^{-6}$

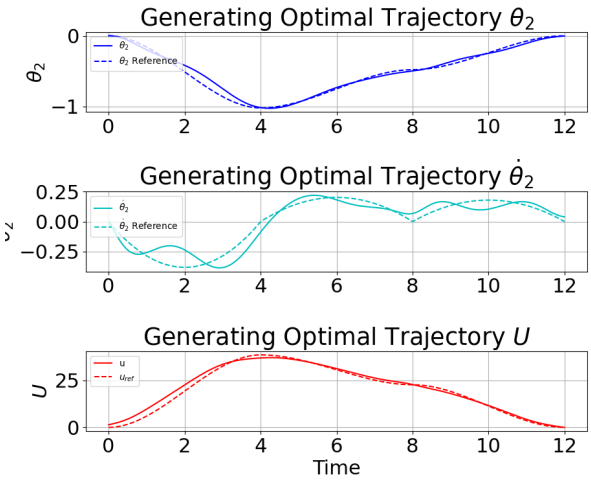
3.2 Results

The cubic polynomial generates a smooth reference path, as shown as dashed lines in Figure 3.1. The Newton method tries to follow the reference by

minimizing the cost value as described previously. The algorithm takes nine iterations, and the smooth reference path assists the algorithm in converging to each intermediate point, resulting in a cost value with a significantly lower error and cost value, as shown in Figure 3.2. The tracking performance improves significantly since the smooth reference is generated by respecting the system's dynamics.

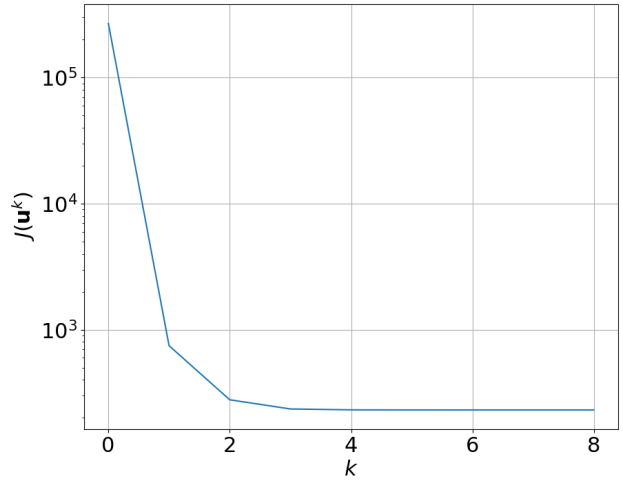


(a) q_1 trajectory plot

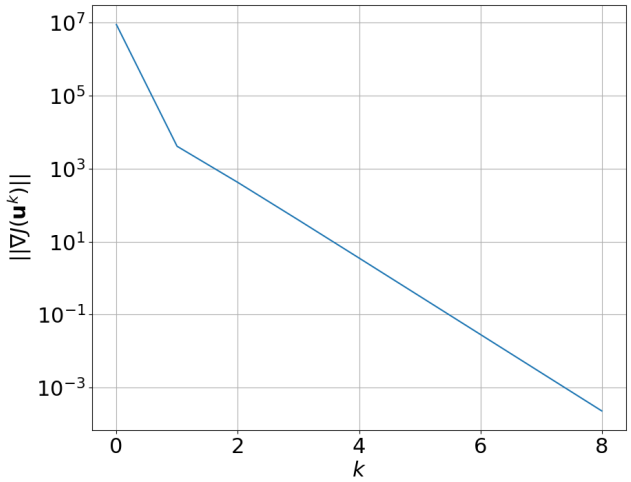


(b) q_2 trajectory plot

Figure 3.1: Trajectory generation with smooth reference



(a) Cost function



(b) Descent direction

Figure 3.2: Cost function over k iterations with smooth reference

Chapter 4

Task 3: Trajectory Tracking via LQR

4.1 Trajectory Tracking Formulation

By linearizing the flexible arm dynamics around the optimal trajectory $(\mathbf{x}^*, \mathbf{u}^*)$ computed in Task 2, we can design an optimal feedback controller to track this reference.

Now, we use the Linear Quadratic Regulator (LQR) in this task. This involves solving the following time-varying Linear Quadratic (LQ) optimization problem:

$$\min_{\substack{\Delta x_1,\ldots,\Delta x_T \\ \Delta u_0,\ldots,\Delta u_{T-1}}} \sum_{t=0}^{T-1} \left(\Delta x_t^\top Q_t^{\text{reg}} \Delta x_t + \Delta u_t^\top R_t^{\text{reg}} \Delta u_t \right) + \Delta x_T^\top Q_T^{\text{reg}} \Delta x_T \tag{4.1}$$

subject to the linearized dynamics:

$$\Delta x_{t+1} = A_t^{\text{opt}} \Delta x_t + B_t^{\text{opt}} \Delta u_t, \quad t = 0, \ldots, T-1, \tag{4.2}$$

with the initial condition $\Delta x_0 = 0$.
The cost matrices are positive semi-definite and positive definite, respectively:

$$Q_t \succeq 0, \quad Q_t \in \mathbb{R}^{n \times n}, \quad R_t \succ 0, \quad R_t \in \mathbb{R}^{m \times m}, \quad Q_T \succeq 0, \quad Q_T \in \mathbb{R}^{n \times n}.$$

4.1.1 System Linearization

The linearization of the nonlinear system dynamics about the optimal trajectory is implemented in `dynamics.py`. The dynamics are expressed as:

$$\Delta x_{t+1} = A_t^{\text{opt}} \Delta x_t + B_t^{\text{opt}} \Delta u_t, \tag{4.3}$$

where the state transition matrix A_t^{opt} and input matrix B_t^{opt} represent the Jacobians of the system dynamics evaluated at the optimal operating point $(\mathbf{x}_t^*, \mathbf{u}_t^*)$.

4.1.2 Riccati Recursion

The sequence of Riccati matrices P_t are computed backward in time starting from the terminal condition:

$$P_T = Q_T.$$

For $t = T-1, \ldots, 0$, we solve the Difference Riccati Equation (DRE):

$$P_t = Q_t + A_t^{\text{opt}\top} P_{t+1} A_t^{\text{opt}} - A_t^{\text{opt}\top} P_{t+1} B_t^{\text{opt}} \left(R_t + B_t^{\text{opt}\top} P_{t+1} B_t^{\text{opt}} \right)^{-1} B_t^{\text{opt}\top} P_{t+1} A_t^{\text{opt}}. \tag{4.4}$$

The optimal feedback gain matrix K_t is then defined as:

$$K_t := -\left(R_t + B_t^{\text{opt}\top} P_{t+1} B_t^{\text{opt}} \right)^{-1} B_t^{\text{opt}\top} P_{t+1} A_t^{\text{opt}}. \tag{4.5}$$

4.1.3 Feedback Control Law

Using the computed gains, the optimal reference trajectory can be tracked by applying the following feedback law to the closed-loop system:

$$u_t^{\text{tracked}} = u_t^{\text{opt}} + K_t \left(x_t^{\text{tracked}} - x_t^* \right), \tag{4.6}$$

$$x_{t+1}^{\text{tracked}} = f\left(x_t^{\text{tracked}}, u_t^{\text{tracked}} \right). \tag{4.7}$$

This process yields the actual tracked trajectory $(\mathbf{x}^{\text{tracked}}, \mathbf{u}^{\text{tracked}})$.

4.1.4 Backward Recursion Algorithm

The implementation details of the backward pass for $t = T-1$ down to 0 are as follows:

$$M_t = R_t + B_t^\top P_{t+1} B_t, \tag{4.8}$$

$$m_t = r_t + B_t^\top p_{t+1}, \tag{4.9}$$

$$K_t = -M_t^{-1} \left(B_t^\top P_{t+1} A_t + S_t \right), \tag{4.10}$$

$$\sigma_t = -M_t^{-1} m_t, \tag{4.11}$$

$$P_t = Q_t + A_t^\top P_{t+1} A_t - \left(B_t^\top P_{t+1} A_t + S_t \right)^\top M_t^{-1} \left(B_t^\top P_{t+1} A_t + S_t \right), \tag{4.12}$$

$$p_t = q_t + A_t^\top p_{t+1} - \left(B_t^\top P_{t+1} A_t + S_t \right)^\top M_t^{-1} m_t. \tag{4.13}$$

Note: If affine terms are not present in the cost function, only the quadratic terms P_t and K_t are computed.

4.1.5 Forward Simulation

Once the policy parameters are computed, the forward simulation is performed using:

$$u_t = K_t x_t + \sigma_t, \tag{4.14}$$

$$x_{t+1} = A_t x_t + B_t u_t. \tag{4.15}$$

Numerical Stability: To ensure numerical robustness, the algorithm verifies that the matrix M_t remains positive definite at each time step. If numerical issues arise, a regularization step is applied:

$$M_t \leftarrow M_t + \epsilon I,$$

where $\epsilon > 0$ is a small regularization factor.

This implementation provides a versatile LQR solver capable of handling time-varying systems and affine costs, utilizing Riccati recursion to compute optimal gains and simulate the resulting trajectory.

4.2 LQR Tracking Results

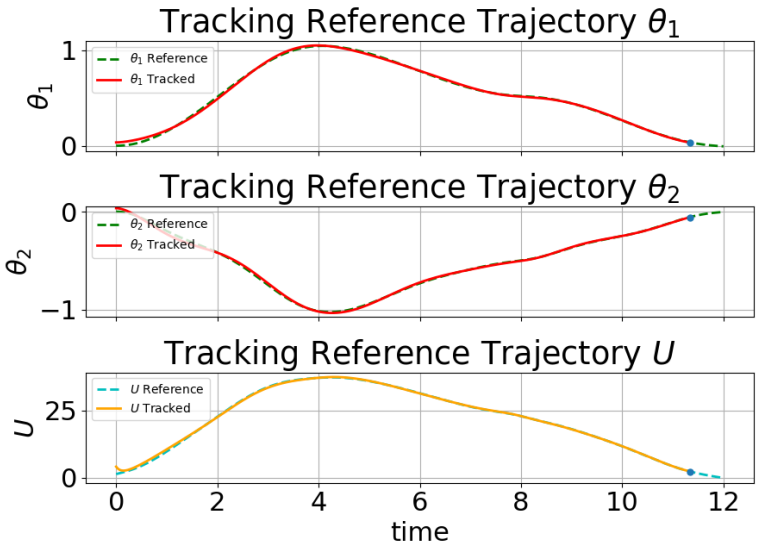


Figure 4.1: LQR tracking performance for reference trajectories θ_1 and θ_2 under initial perturbation.

4.3 Tracking Error Analysis

To quantify the performance of the controller, we define the tracking error as the deviation between the tracked trajectory and the desired optimal

trajectory:

$$e_t := x_t^{\text{tracked}} - x_t^\star = \begin{bmatrix} e_{\theta_1}(t) & e_{\theta_2}(t) & e_{\dot{\theta}_1}(t) & e_{\dot{\theta}_2}(t) \end{bmatrix}^\top . \tag{4.16}$$

Figure 4.2 illustrates the evolution of e_t over time given perturbed initial conditions.

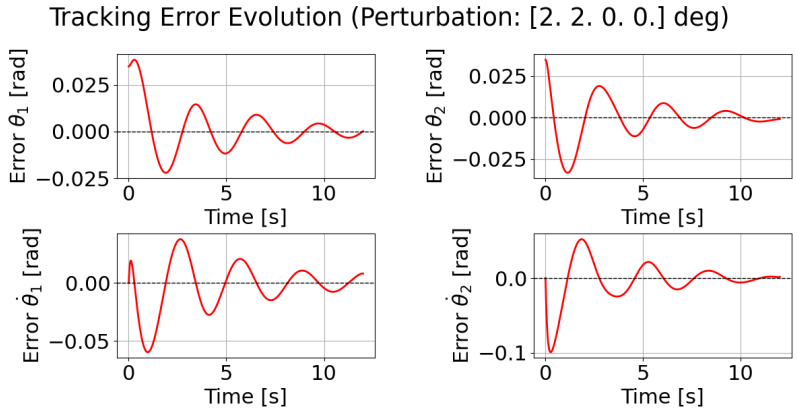


Figure 4.2: Evolution of the tracking error $e_t = x_t^{\text{tracked}} - x_t^\star$.

Chapter 5

Task 4: Trajectory Tracking via MPC

5.1 Model Predictive Control Framework

Tracking a trajectory via Model Predictive Control (MPC) is an advanced control strategy that enables a system to follow a reference path by continuously predicting and optimizing future control inputs over a finite time horizon. A primary advantage of MPC over LQR is its inherent ability to enforce state and input constraints explicitly.

The Receding Horizon principle is applied as follows: at each time instant t, the current state x_t is measured. The controller then computes the optimal future trajectory:

$$\mathbf{x}^*_{t|t}, \ldots, \mathbf{x}^*_{t+T|t}, \quad \mathbf{u}^*_{t|t}, \ldots, \mathbf{u}^*_{t+T-1|t}$$

initialized at $\mathbf{x}_t = \mathbf{x}^{\text{meas}}_t$. Only the first control input $\mathbf{u}^*_{t|t}$ is applied to the system. The state is then measured at $t+1$, and the optimization is repeated.

5.1.1 Optimization Problem Formulation

At each time step t, we solve the following constrained optimization problem:

$$\min_{\substack{x_t,\ldots,x_{t+T} \\ u_t,\ldots,u_{t+T-1}}} \sum_{\lambda=t}^{t+T-1} \left(x_\lambda^\top Q_\lambda x_\lambda + u_\lambda^\top R_\lambda u_\lambda \right) + x_{t+T}^\top Q_T x_{t+T} \tag{5.1}$$

subject to:

$$x_{\lambda+1} = A_\lambda^{\text{opt}} x_\lambda + B_\lambda^{\text{opt}} u_\lambda, \quad \lambda = t, \ldots, t+T-1, \tag{5.2}$$

$$x_t = x_t^{\text{meas}}, \tag{5.3}$$

$$x_\lambda \in \mathcal{X}, \quad u_\lambda \in \mathcal{U}, \tag{5.4}$$

where T represents the prediction horizon.

5.1.2 Implementation Details

The function `linear_mpc` implements the MPC strategy utilizing the `cvxpy` optimization library. The objective is to minimize the deviation of the state x_t and control input u_t from the reference trajectories $x_t^\star$ and $u_t^\star$, incorporating a terminal cost for stability:

$$\begin{aligned}\min_{\{x_t,u_t\}} \sum_{t=0}^{T_{\text{pred}}-2} &\Big[(x_t - x_t^\star)^\top Q(x_t - x_t^\star) + (u_t - u_t^\star)^\top R(u_t - u_t^\star)\Big] \\ &+(x_{T_{\text{pred}}-1} - x^\star_{T_{\text{pred}}-1})^\top Q_f(x_{T_{\text{pred}}-1} - x^\star_{T_{\text{pred}}-1})\end{aligned} \tag{5.5}$$

The weighting matrices are defined as:

- $Q \succeq 0$: State weighting matrix (stage cost).
- $R \succ 0$: Control input weighting matrix (stage cost).
- $Q_f \succeq 0$: Terminal state weighting matrix (terminal cost).

Here, $\{x_t^\star\}$ and $\{u_t^\star\}$ denote the desired reference trajectories for the state and input, respectively.

5.1.3 Constraints

The optimization is subject to the following set of constraints:

1. **System Dynamics:**
$$x_{t+1} = A_t x_t + B_t u_t, \quad \forall t = 0, \ldots, T_{\text{pred}} - 2 \tag{5.6}$$
2. **Initial Condition:**
$$x_0 = x_t \tag{5.7}$$
3. **Input Saturation:**
$$u_{\min} \leq u_t \leq u_{\max}, \quad \forall t \tag{5.8}$$
4. **State constraints (if applicable):**
$$x_{1,\min} \leq x_{1,t} \leq x_{1,\max}, \quad \forall t \tag{5.9}$$
$$x_{2,\min} \leq x_{2,t} \leq x_{2,\max}, \quad \forall t \tag{5.10}$$

5.1.4 Optimization Output

Upon convergence, the solver returns:

- $u_{\text{opt}}^{\text{mpc}} \in \mathbb{R}^{n_i}$: The optimal control input u_{opt}^0 for the current time step.
- $xx_{\text{traj}}^{\text{mpc}}$: The predicted optimal state trajectory $\{x_t^{\text{opt}}\}_{t=0}^{T_{\text{pred}}-1}$.
- $uu_{\text{traj}}^{\text{mpc}}$: The predicted optimal control input sequence $\{u_t^{\text{opt}}\}_{t=0}^{T_{\text{pred}}-1}$.

5.2 MPC Tracking Results: Unconstrained

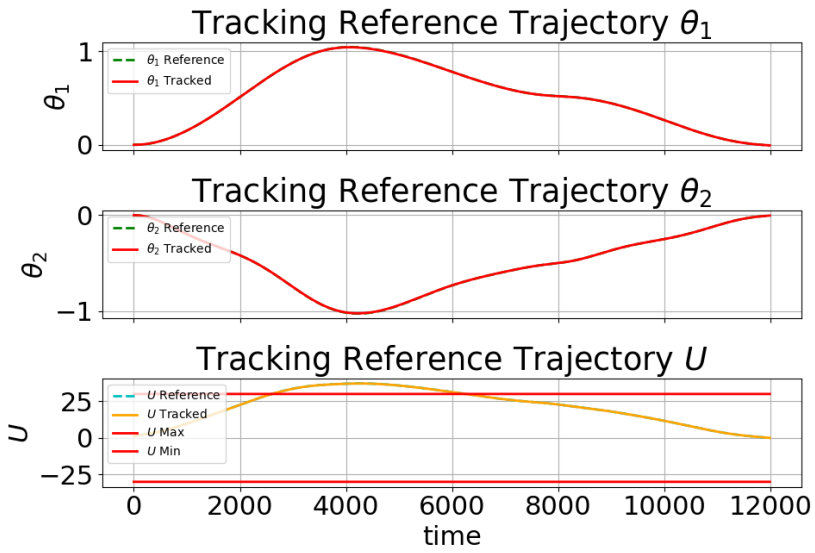


Figure 5.1: MPC tracking performance for θ_1 and θ_2 with zero initial condition (no constraints).

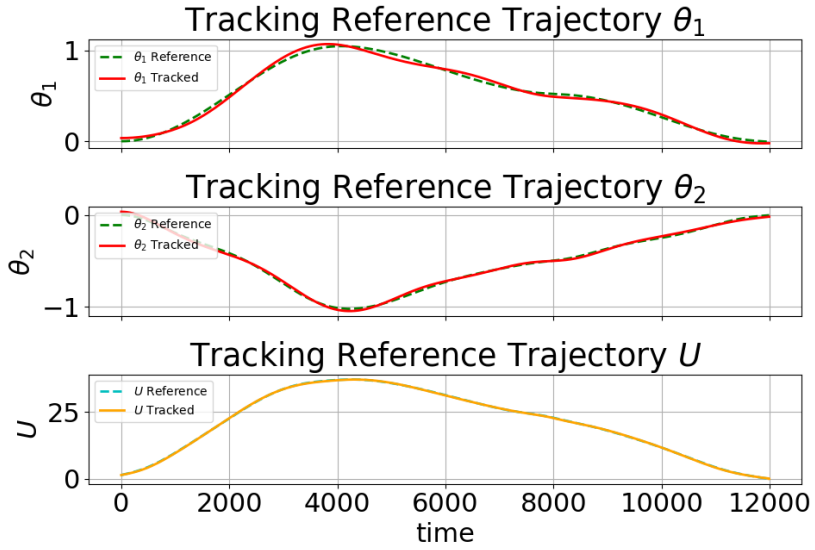


Figure 5.2: MPC tracking performance for θ_1 and θ_2 with perturbed initial condition (no constraints).

5.3 MPC Tracking Results: Constrained

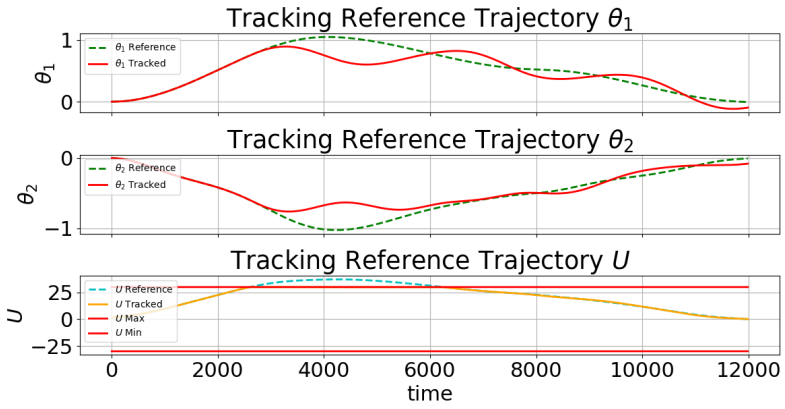


Figure 5.3: MPC tracking performance for θ_1 and θ_2 subject to input constraints.

5.4 MPC Tracking Error

Similar to the LQR analysis, the MPC tracking error is defined as:

$$e_t := x_t^{\text{tracked}} - x_t^{\star}.$$

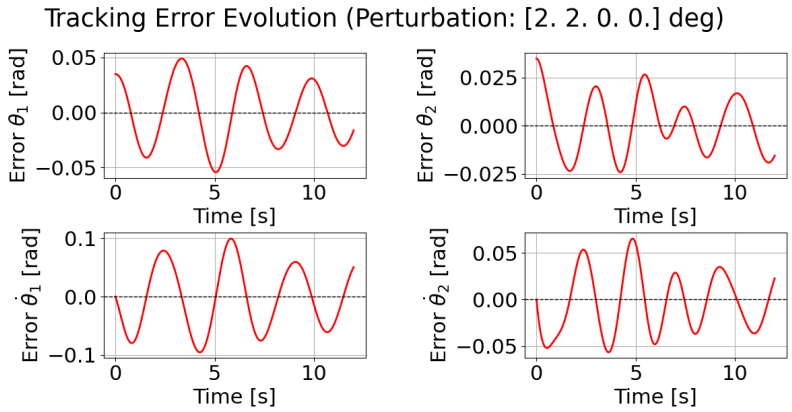


Figure 5.4: Evolution of the MPC tracking error (only with perturbation of 2 deg) $e_t = x_t^{\text{tracked}} - x_t^{\star}$.

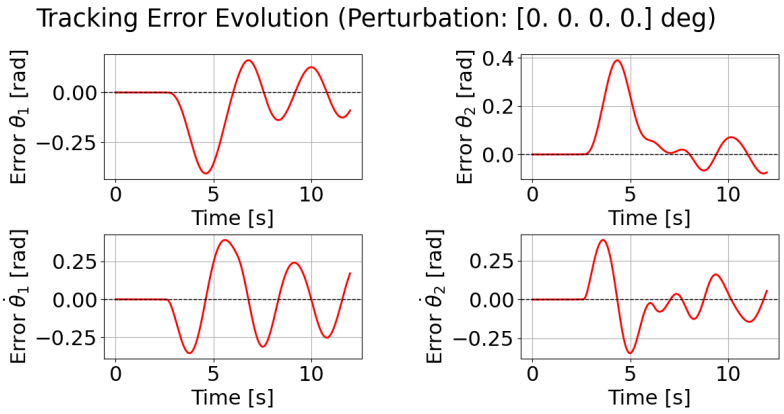


Figure 5.5: Evolution of the MPC tracking error (only with very tight input constraint)

Chapter 6

Task 5: Animation and Visualization

6.1 Animation Framework

To validate the physical feasibility of the generated trajectories, we utilized the `animation.py` module. This tool renders the motion of the flexible robotic arm in a 2D plane using the `matplotlib.animation` library.

The animation function `animate_flexible_robot_arm` visualizes:

- **Robot Geometry:** (Set 2 parameters).
- **Optimal Path (Blue):** The trajectory computed by the Newton optimization or the actual path tracked by the controller.
- **Reference Path (Green):** The desired target path for comparison.

6.2 Observations

The animation provides a crucial qualitative check. It visually confirms that the "Smooth Reference" generated in Task 2 results in a fluid motion of the arm, whereas the "Step Reference" from Task 1 demands physically impossible instantaneous acceleration. Furthermore, the tracking animation confirms that the LQR and MPC controllers successfully pull the robot back to the reference path after the initial perturbation.

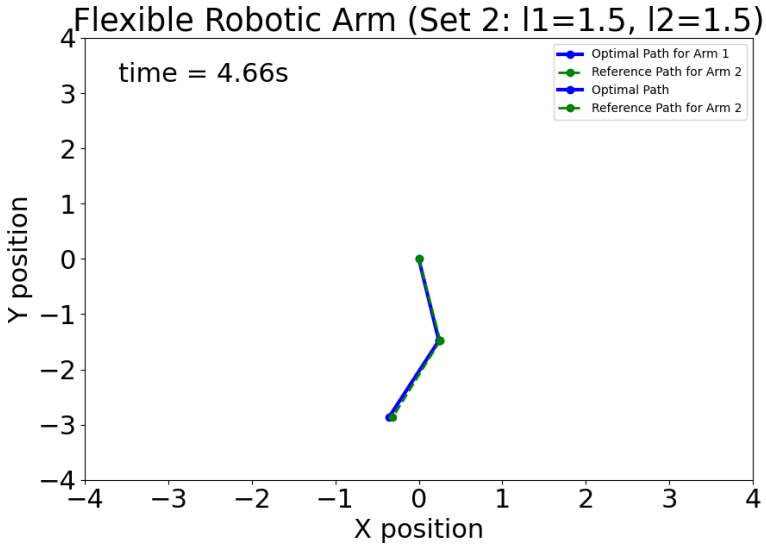


Figure 6.1: Snapshot of the animation interface showing the flexible robotic arm (blue solid line) tracking the reference trajectory (green dashed line).

Conclusions

This project presented a complete optimal control framework for a flexible robotic link with nonlinear dynamics.The system was discretized using RK4, and equilibrium configurations were used to define consistent reference trajectories. An optimal transition between equilibria was computed via a Newton-based optimal control method with Armijo line search, ensuring robust convergence.

The resulting optimal trajectory was linearized to design feedback controllers for trajectory tracking. A Linear Quadratic Regulator (LQR) achieved accurate tracking and robustness to initial condition perturbations, while a Model Predictive Control (MPC) strategy further improved performance by explicitly handling input constraints. Overall, the results demonstrate the effectiveness of combining nonlinear optimal control with linear feedback and predictive control techniques for trajectory generation and tracking in flexible robotic systems.

Bibliography

[1] D. Bertsekas, *Nonlinear Programming*, Athena Scientific, 2016.

[2] OPTCON Course matreials on UNIBO Virtuale .